SQL LAB EXAM · ANSWERS

Full Tested Solutions

Answers to every question in the 8 schemas from Question Sets 1–3 and the Model Questions, plus procedures, functions, triggers and XML. All tested on MySQL-compatible MariaDB.

1. Student Management
2. Employee Management
3. Railway Reservation
4. Hospital Management
5. Bank Management
6. Airline Reservation
7. Movie Booking
8. Food Delivery
9. Stored Procedures And Functions (Mysql)
10. Triggers (Insert / Update / Delete) In Mysql
11. Xml + Xsd Validation

1. Student Management

```
--
================================================
====
-- 1. STUDENT MANAGEMENT
-- Student(StudentID PK, Name, Dept, City, CGPA)
-- Course(CourseID PK, CourseName, Credits)
-- Enrollment(EnrollID PK, StudentID FK, CourseID FK,
Marks)
--
================================================
====
DROP DATABASE IF EXISTS college;
CREATE DATABASE college;
USE college;

-- Q: Create Student with PRIMARY KEY, NOT NULL,
UNIQUE, CHECK (CGPA BETWEEN 0 AND 10)
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name      VARCHAR(50) NOT NULL UNIQUE,
    Dept      VARCHAR(20) NOT NULL,
    City      VARCHAR(30),
    CGPA      DECIMAL(4,2) CHECK (CGPA BETWEEN 0 AND
10)
);

-- Insert the three sample student records
INSERT INTO Student VALUES (101, 'Arun',  'CSE',
'Chennai',    8.50);
INSERT INTO Student VALUES (102, 'Priya', 'ECE',
'Madurai',    9.10);
INSERT INTO Student VALUES (103, 'Karthik','CSE',
'Chennai',   7.20);
SELECT * FROM Student;

CREATE TABLE Course (
    CourseID   VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(50) NOT NULL,
    Credits    INT
);
INSERT INTO Course VALUES ('C1', 'DBMS', 4), ('C2',
```

```
'Networks', 3);

-- Q: Create Enrollment with foreign keys referencing
Student and Course
CREATE TABLE Enrollment (
    EnrollID  INT PRIMARY KEY,
    StudentID INT,
    CourseID  VARCHAR(10),
    Marks     INT,
    FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
    FOREIGN KEY (CourseID)  REFERENCES
Course(CourseID)
);
-- Insert an enrollment record for StudentID 101 and
CourseID C1
INSERT INTO Enrollment VALUES (1, 101, 'C1', 85);
INSERT INTO Enrollment VALUES (2, 102, 'C2', 78);

-- Display all students from Chennai
SELECT * FROM Student WHERE City = 'Chennai';
-- Average CGPA of CSE students
SELECT AVG(CGPA) AS Avg_CSE_CGPA FROM Student WHERE
Dept = 'CSE';

-- Q: Students whose CGPA > average CGPA of all
students (SUBQUERY)
SELECT Name FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM Student);

-- Q: Insert new student, enroll in a course, COMMIT
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Student VALUES (104, 'Divya', 'IT',
'Coimbatore', 8.80);
INSERT INTO Enrollment VALUES (3, 104, 'C1', 90);
COMMIT;

-- Display student names and course names for all
enrolled students (JOIN)
SELECT s.Name, c.CourseName
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c     ON e.CourseID  = c.CourseID;

-- Q: Update CGPA and insert enrollment, ROLLBACK
both
```

```sql
START TRANSACTION;
UPDATE Student SET CGPA = 9.50 WHERE StudentID = 103;
INSERT INTO Enrollment VALUES (4, 103, 'C2', 70);
ROLLBACK;
SELECT * FROM Student WHERE StudentID = 103;   -- CGPA still 7.20

-- Q: Insert student, SAVEPOINT, insert enrollment, roll back to savepoint
START TRANSACTION;
INSERT INTO Student VALUES (105, 'Rahul', 'MECH', 'Salem', 7.90);
SAVEPOINT sp1;
INSERT INTO Enrollment VALUES (5, 105, 'C1', 65);
ROLLBACK TO sp1;
COMMIT;
SELECT * FROM Student WHERE StudentID = 105;    -- student remains
SELECT * FROM Enrollment WHERE StudentID = 105; -- enrollment gone

-- Q: Update marks, savepoint, update CGPA, rollback to savepoint
START TRANSACTION;
UPDATE Enrollment SET Marks = 95 WHERE EnrollID = 1;
SAVEPOINT before_cgpa;
UPDATE Student SET CGPA = 9.90 WHERE StudentID = 101;
ROLLBACK TO before_cgpa;
COMMIT;                                    -- marks change kept, CGPA change undone

-- Delete an enrollment record and ROLLBACK to restore it
START TRANSACTION;
DELETE FROM Enrollment WHERE EnrollID = 2;
ROLLBACK;
SELECT * FROM Enrollment;
SET autocommit = 1;

-- Q: GRANT / REVOKE
CREATE USER IF NOT EXISTS 'student_user'@'localhost' IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'faculty_user'@'localhost' IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'course_admin'@'localhost' IDENTIFIED BY 'pass123';
GRANT SELECT ON college.Student TO
```

```
'student_user'@'localhost';
GRANT SELECT, INSERT ON college.Enrollment TO
'faculty_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON college.Student TO
'faculty_user'@'localhost';
REVOKE INSERT ON college.Enrollment FROM
'faculty_user'@'localhost';
GRANT ALL PRIVILEGES ON college.Course TO
'course_admin'@'localhost';
SHOW GRANTS FOR 'faculty_user'@'localhost';

-- Q: VIEW + INDEX
CREATE VIEW Student_Course_View AS
SELECT s.StudentID, s.Name, c.CourseName, e.Marks
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c     ON e.CourseID  = c.CourseID;
SELECT * FROM Student_Course_View;

CREATE INDEX idx_student_dept ON Student(Dept);
SHOW INDEX FROM Student;
```

2. Employee Management

```
--
=================================================
====
-- 2. EMPLOYEE MANAGEMENT
-- Employee(EmpID, EmpName, DeptID, Salary, HireDate)
-- Department(DeptID, DeptName, ManagerID)
--
=================================================
====
DROP DATABASE IF EXISTS company;
CREATE DATABASE company;
USE company;

CREATE TABLE Department (
    DeptID    INT PRIMARY KEY,
    DeptName  VARCHAR(30) NOT NULL,
    ManagerID INT
);
CREATE TABLE Employee (
    EmpID    INT PRIMARY KEY,
    EmpName  VARCHAR(50) NOT NULL,
    DeptID   INT,
    Salary   DECIMAL(10,2) CHECK (Salary > 20000),
    HireDate DATE
);
INSERT INTO Department VALUES (1, 'HR', 201), (2,
'IT', 202);

INSERT INTO Employee VALUES (201, 'Ravi',  1, 45000,
'2020-06-15');
INSERT INTO Employee VALUES (202, 'Meena', 2, 60000,
'2019-03-10');
INSERT INTO Employee VALUES (203, 'Suresh',2, 52000,
'2021-01-05');

-- Update Meena's salary to 75000
UPDATE Employee SET Salary = 75000 WHERE EmpName =
'Meena';

-- Add a foreign key from Employee.DeptID to
Department.DeptID
```

```sql
ALTER TABLE Employee
ADD CONSTRAINT fk_dept FOREIGN KEY (DeptID)
REFERENCES Department(DeptID);

-- Try inserting an employee with DeptID 99  -> FAILS
-- INSERT INTO Employee VALUES (205, 'Vijay', 99,
30000, '2022-02-02');
-- ERROR 1452: Cannot add or update a child row: a
foreign key constraint fails.
-- Reason: DeptID 99 does not exist in the parent
table Department
-- (referential integrity).

-- Employees with salary > average salary
SELECT EmpID, EmpName, Salary FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);

-- Departments having more than one employee (GROUP
BY + HAVING)
SELECT DeptID, COUNT(*) AS EmpCount
FROM Employee
GROUP BY DeptID
HAVING COUNT(*) > 1;

-- Highest salary in each department
SELECT DeptID, MAX(Salary) AS HighestSalary
FROM Employee
GROUP BY DeptID;

-- Pairs of employees in the same department (SELF
JOIN)
SELECT e1.EmpName AS Employee1, e2.EmpName AS
Employee2, e1.DeptID
FROM Employee e1
JOIN Employee e2 ON e1.DeptID = e2.DeptID AND
e1.EmpID < e2.EmpID;

-- Increase salary of all employees in a department,
COMMIT
SET autocommit = 0;
START TRANSACTION;
UPDATE Employee SET Salary = Salary + 5000 WHERE
DeptID = 2;
COMMIT;

-- Update salary, savepoint, update department,
rollback to savepoint
```

```sql
START TRANSACTION;
UPDATE Employee SET Salary = 50000 WHERE EmpID = 201;
SAVEPOINT sp1;
UPDATE Employee SET DeptID = 2 WHERE EmpID = 201;
ROLLBACK TO sp1;
COMMIT;

-- Delete an employee and ROLLBACK
START TRANSACTION;
DELETE FROM Employee WHERE EmpID = 203;
ROLLBACK;

-- Insert new department and employee, COMMIT both
START TRANSACTION;
INSERT INTO Department VALUES (3, 'Sales', 204);
INSERT INTO Employee VALUES (204, 'Anil', 3, 35000,
'2023-07-01');
COMMIT;

-- Increase by 10%, savepoint, another 5%, rollback
to first savepoint
START TRANSACTION;
UPDATE Employee SET Salary = Salary * 1.10;
SAVEPOINT after_10;
UPDATE Employee SET Salary = Salary * 1.05;
ROLLBACK TO after_10;
COMMIT;
SET autocommit = 1;
SELECT * FROM Employee;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'hr_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'manager_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'employee_user'@'localhost'
IDENTIFIED BY 'pass123';
GRANT SELECT ON company.Employee TO
'hr_user'@'localhost';
GRANT SELECT, UPDATE ON company.Employee TO
'manager_user'@'localhost';
GRANT INSERT ON company.Employee TO
'hr_user'@'localhost';
REVOKE UPDATE ON company.Employee FROM
'manager_user'@'localhost';
GRANT SELECT ON company.Department TO
'employee_user'@'localhost';
```

```sql
-- VIEW + INDEX
CREATE VIEW Employee_Department_View AS
SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID;
SELECT * FROM Employee_Department_View;

CREATE INDEX idx_employee_salary ON Employee(Salary);
```

3. Railway Reservation

```
--
==========================================================
====
-- 3. RAILWAY RESERVATION
-- Passenger(PassengerID, Name, Age)
-- Train(TrainID, TrainName, Source, Destination)
-- Booking(BookingID, PassengerID, TrainID,
SeatsBooked, Fare)
--
==========================================================
====
DROP DATABASE IF EXISTS railway;
CREATE DATABASE railway;
USE railway;

CREATE TABLE Passenger (
    PassengerID INT PRIMARY KEY,
    Name        VARCHAR(50) NOT NULL,
    Age         INT
);

-- Train with TrainName UNIQUE
CREATE TABLE Train (
    TrainID     INT PRIMARY KEY,
    TrainName   VARCHAR(50) UNIQUE,
    Source      VARCHAR(30),
    Destination VARCHAR(30)
);
INSERT INTO Train VALUES (1, 'Vaigai Express',
'Chennai', 'Madurai');
INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');
-- Delete the train whose destination is Coimbatore
DELETE FROM Train WHERE Destination = 'Coimbatore';
INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');

-- Booking with FKs; ON DELETE CASCADE on TrainID
CREATE TABLE Booking (
    BookingID   INT PRIMARY KEY,
    PassengerID INT,
```

```
    TrainID     INT,
    SeatsBooked INT,
    Fare        DECIMAL(8,2),
    FOREIGN KEY (PassengerID) REFERENCES
Passenger(PassengerID),
    FOREIGN KEY (TrainID) REFERENCES Train(TrainID)
ON DELETE CASCADE
);
-- ON DELETE CASCADE: when a train is deleted from
Train, all bookings
-- in Booking with that TrainID are deleted
automatically.

INSERT INTO Passenger VALUES (1, 'Arun', 25), (2,
'Priya', 30), (3, 'Kumar', 40);
INSERT INTO Booking VALUES (1, 1, 1, 2, 500), (2, 2,
1, 1, 300), (3, 1, 2, 3, 900);

-- Show cascade
DELETE FROM Train WHERE TrainID = 2;
SELECT * FROM Booking;           -- booking 3
disappeared automatically
INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');
INSERT INTO Booking VALUES (3, 1, 2, 3, 900);

-- Passengers who made at least one booking
SELECT Name FROM Passenger
WHERE PassengerID IN (SELECT PassengerID FROM
Booking);

-- All passengers and their bookings incl. those with
no booking (LEFT JOIN)
SELECT p.PassengerID, p.Name, b.BookingID, b.TrainID,
b.Fare
FROM Passenger p
LEFT JOIN Booking b ON p.PassengerID = b.PassengerID;

-- Bookings whose fare > average fare of the SAME
train (CORRELATED subquery)
SELECT * FROM Booking b1
WHERE Fare > (SELECT AVG(Fare) FROM Booking b2 WHERE
b2.TrainID = b1.TrainID);

-- Insert passenger + booking, COMMIT
SET autocommit = 0;
START TRANSACTION;
```

```sql
INSERT INTO Passenger VALUES (4, 'Divya', 22);
INSERT INTO Booking VALUES (4, 4, 1, 2, 600);
COMMIT;

-- Total fare collected for each train
SELECT TrainID, SUM(Fare) AS TotalFare FROM Booking
GROUP BY TrainID;

-- Update seats & fare, ROLLBACK
START TRANSACTION;
UPDATE Booking SET SeatsBooked = 5, Fare = 1500 WHERE
BookingID = 1;
ROLLBACK;

-- Insert passenger, SAVEPOINT, booking, rollback to
savepoint
START TRANSACTION;
INSERT INTO Passenger VALUES (5, 'Rahul', 28);
SAVEPOINT sp1;
INSERT INTO Booking VALUES (5, 5, 2, 1, 450);
ROLLBACK TO sp1;
COMMIT;

-- Delete a booking and ROLLBACK
START TRANSACTION;
DELETE FROM Booking WHERE BookingID = 2;
ROLLBACK;

-- Modify fare, savepoint, modify seats, rollback to
savepoint
START TRANSACTION;
UPDATE Booking SET Fare = 550 WHERE BookingID = 1;
SAVEPOINT sp2;
UPDATE Booking SET SeatsBooked = 4 WHERE BookingID =
1;
ROLLBACK TO sp2;
COMMIT;
SET autocommit = 1;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS
'reservation_user'@'localhost' IDENTIFIED BY
'pass123';
CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'railway_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```sql
GRANT SELECT ON railway.Train TO
'reservation_user'@'localhost';
GRANT SELECT, INSERT ON railway.Booking TO
'booking_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON railway.Passenger TO
'reservation_user'@'localhost';
REVOKE INSERT ON railway.Booking FROM
'booking_user'@'localhost';
GRANT ALL PRIVILEGES ON railway.Booking TO
'railway_admin'@'localhost';

-- VIEW + INDEX
CREATE VIEW Booking_Details_View AS
SELECT b.BookingID, p.Name, t.TrainName,
b.SeatsBooked, b.Fare
FROM Booking b
JOIN Passenger p ON b.PassengerID = p.PassengerID
JOIN Train t     ON b.TrainID     = t.TrainID;
SELECT * FROM Booking_Details_View;

CREATE INDEX idx_train_destination ON
Train(Destination);
```

4. Hospital Management

```
--
==========================================================
====
-- 4. HOSPITAL MANAGEMENT
-- Patient(PatientID, PatientName, City)
-- Doctor(DoctorID, DoctorName, Specialization)
-- Appointment(AppID, PatientID, DoctorID, VisitDate,
Fee)
--
==========================================================
====
DROP DATABASE IF EXISTS hospital;
CREATE DATABASE hospital;
USE hospital;

CREATE TABLE Patient (
    PatientID   INT PRIMARY KEY,
    PatientName VARCHAR(50) NOT NULL,
    City        VARCHAR(30)
);
-- Doctor with Specialization NOT NULL
CREATE TABLE Doctor (
    DoctorID       INT PRIMARY KEY,
    DoctorName     VARCHAR(50) NOT NULL,
    Specialization VARCHAR(40) NOT NULL
);
-- Appointment with FKs to Patient and Doctor
CREATE TABLE Appointment (
    AppID     INT PRIMARY KEY,
    PatientID INT,
    DoctorID  INT,
    VisitDate DATE,
    Fee       DECIMAL(8,2),
    FOREIGN KEY (PatientID) REFERENCES
Patient(PatientID),
    FOREIGN KEY (DoctorID)  REFERENCES
Doctor(DoctorID)
);
INSERT INTO Patient VALUES (1, 'Arun', 'Chennai'),
(2, 'Priya', 'Madurai');
INSERT INTO Doctor VALUES (1, 'Dr. Ravi',
```

```
'Cardiology'), (2, 'Dr. Meena', 'Neurology'),
                          (3, 'Dr. Kumar', 'Ortho');
INSERT INTO Appointment VALUES (1, 1, 1, '2025-01-
10', 500),
                               (2, 2, 1, '2025-01-
11', 500),
                               (3, 1, 2, '2025-01-
12', 800);

-- Doctors who have at least one appointment
SELECT * FROM Doctor
WHERE DoctorID IN (SELECT DoctorID FROM Appointment);

-- Count appointments handled by each doctor
SELECT d.DoctorName, COUNT(a.AppID) AS
TotalAppointments
FROM Doctor d
LEFT JOIN Appointment a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.DoctorName;

-- Patient names and doctor names for all
appointments
SELECT p.PatientName, d.DoctorName
FROM Appointment a
JOIN Patient p ON a.PatientID = p.PatientID
JOIN Doctor d  ON a.DoctorID  = d.DoctorID;

-- Insert patient + appointment, COMMIT; update fee,
ROLLBACK
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Patient VALUES (3, 'Kavya', 'Salem');
INSERT INTO Appointment VALUES (4, 3, 3, '2025-02-
01', 600);
COMMIT;
START TRANSACTION;
UPDATE Appointment SET Fee = 1000 WHERE AppID = 4;
ROLLBACK;

-- Insert patient, savepoint, insert appointment,
rollback to savepoint
START TRANSACTION;
INSERT INTO Patient VALUES (4, 'Rahul', 'Trichy');
SAVEPOINT sp1;
INSERT INTO Appointment VALUES (5, 4, 2, '2025-02-
05', 800);
ROLLBACK TO sp1;
```

```sql
COMMIT;

-- Delete appointment and ROLLBACK
START TRANSACTION;
DELETE FROM Appointment WHERE AppID = 3;
ROLLBACK;

-- Update specialization, savepoint, update fee,
rollback to savepoint
START TRANSACTION;
UPDATE Doctor SET Specialization = 'Cardiac Surgery'
WHERE DoctorID = 1;
SAVEPOINT sp2;
UPDATE Appointment SET Fee = 900 WHERE AppID = 1;
ROLLBACK TO sp2;
COMMIT;
SET autocommit = 1;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'receptionist'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS
'hospital_staff'@'localhost' IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'doctor_user'@'localhost'
IDENTIFIED BY 'pass123';
GRANT SELECT ON hospital.Doctor TO
'receptionist'@'localhost';
GRANT SELECT, INSERT ON hospital.Appointment TO
'receptionist'@'localhost';
GRANT SELECT, INSERT, UPDATE ON hospital.Patient TO
'hospital_staff'@'localhost';
REVOKE INSERT ON hospital.Appointment FROM
'receptionist'@'localhost';
GRANT SELECT ON hospital.Doctor TO
'doctor_user'@'localhost';

-- VIEW + INDEX
CREATE VIEW Appointment_Details_View AS
SELECT a.AppID, p.PatientName, d.DoctorName,
d.Specialization, a.VisitDate, a.Fee
FROM Appointment a
JOIN Patient p ON a.PatientID = p.PatientID
JOIN Doctor d  ON a.DoctorID  = d.DoctorID;
SELECT * FROM Appointment_Details_View;
```

```
CREATE INDEX idx_doctor_specialization ON
Doctor(Specialization);
```

5. Bank Management

```
--
================================================
====
-- 5. BANK MANAGEMENT
-- Customer(CustomerID, CustomerName, City)
-- Account(AccountNo, CustomerID, AccountType,
Balance)
-- Transaction(TxnID, AccountNo, Amount, TxnType)
--
================================================
====
DROP DATABASE IF EXISTS bank;
CREATE DATABASE bank;
USE bank;

CREATE TABLE Customer (
    CustomerID   INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    City         VARCHAR(30)
);
-- Account with CHECK (Balance >= 1000)
CREATE TABLE Account (
    AccountNo   INT PRIMARY KEY,
    CustomerID  INT,
    AccountType VARCHAR(20),
    Balance     DECIMAL(12,2) CHECK (Balance >= 1000)
);
-- Add FK from Account.CustomerID to
Customer.CustomerID
ALTER TABLE Account
ADD CONSTRAINT fk_cust FOREIGN KEY (CustomerID)
REFERENCES Customer(CustomerID);

-- "Transaction" is a keyword, so wrap it in
backticks to be safe
CREATE TABLE `Transaction` (
    TxnID     INT PRIMARY KEY,
    AccountNo INT,
    Amount    DECIMAL(12,2),
    TxnType   VARCHAR(10),
    FOREIGN KEY (AccountNo) REFERENCES
```

```sql
Account(AccountNo)
);

INSERT INTO Customer VALUES (1, 'Arun', 'Chennai'),
(2, 'Priya', 'Madurai'), (3, 'Kumar', 'Salem');
INSERT INTO Account VALUES (1001, 1, 'Savings',
50000), (1002, 2, 'Current', 80000),
                            (1003, 3, 'Savings',
20000);
INSERT INTO `Transaction` VALUES (1, 1001, 5000,
'Credit');

-- Accounts with balance > average balance
SELECT AccountNo, CustomerID, Balance FROM Account
WHERE Balance > (SELECT AVG(Balance) FROM Account);

-- Customers whose balance > average balance
SELECT c.CustomerName, a.Balance
FROM Customer c
JOIN Account a ON c.CustomerID = a.CustomerID
WHERE a.Balance > (SELECT AVG(Balance) FROM Account);

-- Accounts with balance = maximum balance
SELECT * FROM Account
WHERE Balance = (SELECT MAX(Balance) FROM Account);

-- New customer + account, COMMIT
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Customer VALUES (4, 'Divya', 'Trichy');
INSERT INTO Account VALUES (1004, 4, 'Savings',
15000);
COMMIT;

-- Deduct 5000 and record the transaction, COMMIT
START TRANSACTION;
UPDATE Account SET Balance = Balance - 5000 WHERE
AccountNo = 1001;
INSERT INTO `Transaction` VALUES (2, 1001, 5000,
'Debit');
COMMIT;

-- Transfer 10000 from 1002 to 1003, record both;
ROLLBACK if anything fails
START TRANSACTION;
UPDATE Account SET Balance = Balance - 10000 WHERE
AccountNo = 1002;
```

```
UPDATE Account SET Balance = Balance + 10000 WHERE
AccountNo = 1003;
INSERT INTO `Transaction` VALUES (3, 1002, 10000,
'Debit');
INSERT INTO `Transaction` VALUES (4, 1003, 10000,
'Credit');
COMMIT;      -- if any statement gave an error, run
ROLLBACK; instead

-- Deduct, savepoint, insert transaction, rollback to
savepoint
START TRANSACTION;
UPDATE Account SET Balance = Balance - 2000 WHERE
AccountNo = 1003;
SAVEPOINT sp1;
INSERT INTO `Transaction` VALUES (5, 1003, 2000,
'Debit');
ROLLBACK TO sp1;
COMMIT;

-- Delete a transaction and ROLLBACK
START TRANSACTION;
DELETE FROM `Transaction` WHERE TxnID = 1;
ROLLBACK;
SET autocommit = 1;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'bank_clerk'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'bank_manager'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'bank_admin'@'localhost'
IDENTIFIED BY 'pass123';
GRANT SELECT ON bank.Account TO
'bank_clerk'@'localhost';
GRANT SELECT, INSERT ON bank.`Transaction` TO
'bank_clerk'@'localhost';
GRANT SELECT, UPDATE ON bank.Account TO
'bank_manager'@'localhost';
GRANT UPDATE ON bank.Account TO
'bank_clerk'@'localhost';
REVOKE UPDATE ON bank.Account FROM
'bank_clerk'@'localhost';
GRANT ALL PRIVILEGES ON bank.Account TO
'bank_admin'@'localhost';

-- VIEW + INDEX
```

```
CREATE VIEW Customer_Account_View AS
SELECT c.CustomerID, c.CustomerName, a.AccountNo,
a.AccountType, a.Balance
FROM Customer c
JOIN Account a ON c.CustomerID = a.CustomerID;
SELECT * FROM Customer_Account_View;

CREATE INDEX idx_account_balance ON Account(Balance);
```

6. Airline Reservation

```
--
==========================================================
====
-- 6. AIRLINE RESERVATION
-- Passenger(PassengerID, Name)
-- Flight(FlightID, FlightName, Source, Destination,
TicketPrice)
-- Ticket(TicketID, PassengerID, FlightID, SeatNo)
--
==========================================================
====
DROP DATABASE IF EXISTS airline;
CREATE DATABASE airline;
USE airline;

CREATE TABLE Passenger (
    PassengerID INT PRIMARY KEY,
    Name        VARCHAR(50) NOT NULL
);
CREATE TABLE Flight (
    FlightID    VARCHAR(10) PRIMARY KEY,
    FlightName  VARCHAR(50),
    Source      VARCHAR(30),
    Destination VARCHAR(30),
    TicketPrice DECIMAL(8,2)
);
-- Insert two flights, add 500 to F1's price
INSERT INTO Flight VALUES ('F1', 'IndiGo 6E201',
'Chennai', 'Delhi',  5500);
INSERT INTO Flight VALUES ('F2', 'Air India 440',
'Chennai', 'Mumbai', 4200);
UPDATE Flight SET TicketPrice = TicketPrice + 500
WHERE FlightID = 'F1';

-- Ticket with FKs to Passenger and Flight
CREATE TABLE Ticket (
    TicketID    INT PRIMARY KEY,
    PassengerID INT,
    FlightID    VARCHAR(10),
    SeatNo      VARCHAR(5),
    FOREIGN KEY (PassengerID) REFERENCES
```

```
Passenger(PassengerID),
    FOREIGN KEY (FlightID)    REFERENCES
Flight(FlightID)
);
INSERT INTO Passenger VALUES (1, 'Arun'), (2,
'Priya');

-- Flights by ticket price, descending
SELECT * FROM Flight ORDER BY TicketPrice DESC;

-- All possible passenger-flight combinations (CROSS
JOIN)
SELECT p.Name, f.FlightName
FROM Passenger p CROSS JOIN Flight f;

-- Insert passenger + ticket, COMMIT; update seat,
ROLLBACK
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Passenger VALUES (3, 'Kumar');
INSERT INTO Ticket VALUES (1, 3, 'F1', '12A');
COMMIT;
START TRANSACTION;
UPDATE Ticket SET SeatNo = '20C' WHERE TicketID = 1;
ROLLBACK;

-- Insert passenger, savepoint, book ticket, rollback
to savepoint
START TRANSACTION;
INSERT INTO Passenger VALUES (4, 'Divya');
SAVEPOINT sp1;
INSERT INTO Ticket VALUES (2, 4, 'F2', '5B');
ROLLBACK TO sp1;
COMMIT;

-- Delete ticket and ROLLBACK
START TRANSACTION;
DELETE FROM Ticket WHERE TicketID = 1;
ROLLBACK;

-- Book two tickets, savepoint after first, rollback
the second
START TRANSACTION;
INSERT INTO Ticket VALUES (3, 1, 'F1', '1A');
SAVEPOINT after_first;
INSERT INTO Ticket VALUES (4, 2, 'F2', '2B');
ROLLBACK TO after_first;
```

```
COMMIT;
SET autocommit = 1;
SELECT * FROM Ticket;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'flight_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS
'reservation_user'@'localhost' IDENTIFIED BY
'pass123';
CREATE USER IF NOT EXISTS 'airline_admin'@'localhost'
IDENTIFIED BY 'pass123';
GRANT SELECT ON airline.Flight TO
'flight_user'@'localhost';
GRANT SELECT, INSERT ON airline.Ticket TO
'booking_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON airline.Passenger TO
'reservation_user'@'localhost';
REVOKE INSERT ON airline.Ticket FROM
'booking_user'@'localhost';
GRANT ALL PRIVILEGES ON airline.Ticket TO
'airline_admin'@'localhost';

-- VIEW + INDEX
CREATE VIEW Ticket_Details_View AS
SELECT t.TicketID, p.Name, f.FlightName, f.Source,
f.Destination, t.SeatNo
FROM Ticket t
JOIN Passenger p ON t.PassengerID = p.PassengerID
JOIN Flight f    ON t.FlightID    = f.FlightID;
SELECT * FROM Ticket_Details_View;

CREATE INDEX idx_flight_source ON Flight(Source);
```

7. Movie Booking

```
--
==========================================================
====
-- 7. MOVIE BOOKING
-- Movie(MovieID, MovieName, Genre, TicketPrice)
-- Customer(CustomerID, CustomerName)
-- Booking(BookingID, CustomerID, MovieID,
SeatsBooked)
--
==========================================================
====
DROP DATABASE IF EXISTS cinema;
CREATE DATABASE cinema;
USE cinema;

-- Movie with TicketPrice DEFAULT 150
CREATE TABLE Movie (
    MovieID     INT PRIMARY KEY,
    MovieName   VARCHAR(50) NOT NULL,
    Genre       VARCHAR(20),
    TicketPrice DECIMAL(6,2) DEFAULT 150
);
INSERT INTO Movie VALUES (1, 'Leo', 'Action', 200);
-- No ticket price given -> DEFAULT 150 is used
INSERT INTO Movie (MovieID, MovieName, Genre) VALUES
(2, 'Jailer', 'Action');
INSERT INTO Movie (MovieID, MovieName, Genre) VALUES
(3, 'Vikram', 'Thriller');
SELECT * FROM Movie;

CREATE TABLE Customer (
    CustomerID   INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL
);
CREATE TABLE Booking (
    BookingID   INT PRIMARY KEY,
    CustomerID  INT,
    MovieID     INT,
    SeatsBooked INT,
    FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID),
```

```
    FOREIGN KEY (MovieID)    REFERENCES
Movie(MovieID)
);
INSERT INTO Customer VALUES (1, 'Arun'), (2,
'Priya');
INSERT INTO Booking VALUES (1, 1, 1, 4), (2, 2, 1,
5), (3, 1, 2, 2);

-- Average seats per movie, only where avg > 3
SELECT MovieID, AVG(SeatsBooked) AS AvgSeats
FROM Booking
GROUP BY MovieID
HAVING AVG(SeatsBooked) > 3;

-- Every booking and every movie (FULL OUTER JOIN =
LEFT JOIN UNION RIGHT JOIN in MySQL)
SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
FROM Booking b LEFT JOIN Movie m ON b.MovieID =
m.MovieID
UNION
SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
FROM Booking b RIGHT JOIN Movie m ON b.MovieID =
m.MovieID;

-- New customer + booking, COMMIT; update seats,
ROLLBACK
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Customer VALUES (3, 'Kumar');
INSERT INTO Booking VALUES (4, 3, 3, 3);
COMMIT;
START TRANSACTION;
UPDATE Booking SET SeatsBooked = 10 WHERE BookingID =
4;
ROLLBACK;

-- Booking, savepoint, modify seats, rollback to
savepoint
START TRANSACTION;
INSERT INTO Booking VALUES (5, 2, 3, 2);
SAVEPOINT sp1;
UPDATE Booking SET SeatsBooked = 6 WHERE BookingID =
5;
ROLLBACK TO sp1;
COMMIT;
```

```sql
-- Delete booking and ROLLBACK
START TRANSACTION;
DELETE FROM Booking WHERE BookingID = 5;
ROLLBACK;

-- Two bookings, savepoint between, roll back only
the second
START TRANSACTION;
INSERT INTO Booking VALUES (6, 1, 3, 1);
SAVEPOINT between_bookings;
INSERT INTO Booking VALUES (7, 2, 2, 2);
ROLLBACK TO between_bookings;
COMMIT;
SET autocommit = 1;
SELECT * FROM Booking;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'movie_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'customer_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'movie_admin'@'localhost'
IDENTIFIED BY 'pass123';
GRANT SELECT ON cinema.Movie TO
'movie_user'@'localhost';
GRANT SELECT, INSERT ON cinema.Booking TO
'booking_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON cinema.Customer TO
'customer_user'@'localhost';
REVOKE INSERT ON cinema.Booking FROM
'booking_user'@'localhost';
GRANT ALL PRIVILEGES ON cinema.Booking TO
'movie_admin'@'localhost';

-- VIEW + INDEX
CREATE VIEW Movie_Booking_View AS
SELECT b.BookingID, c.CustomerName, m.MovieName,
m.Genre, b.SeatsBooked
FROM Booking b
JOIN Customer c ON b.CustomerID = c.CustomerID
JOIN Movie m    ON b.MovieID    = m.MovieID;
SELECT * FROM Movie_Booking_View;
```

```sql
CREATE INDEX idx_movie_genre ON Movie(Genre);
```

8. Food Delivery

```
--
==========================================================
====
-- 8. FOOD DELIVERY
-- Restaurant(RestaurantID, RestaurantName, City)
-- Customer(CustomerID, CustomerName, City)
-- FoodOrder(OrderID, CustomerID, RestaurantID,
Quantity, Price, Rating)
--
==========================================================
====
DROP DATABASE IF EXISTS food;
CREATE DATABASE food;
USE food;

CREATE TABLE Restaurant (
    RestaurantID   INT PRIMARY KEY,
    RestaurantName VARCHAR(50) NOT NULL,
    City           VARCHAR(30)
);
CREATE TABLE Customer (
    CustomerID   INT PRIMARY KEY,
    CustomerName VARCHAR(50) NOT NULL,
    City         VARCHAR(30)
);
-- FoodOrder with FKs and ON UPDATE CASCADE
CREATE TABLE FoodOrder (
    OrderID      INT PRIMARY KEY,
    CustomerID   INT,
    RestaurantID INT,
    Quantity     INT,
    Price        DECIMAL(8,2),
    Rating       DECIMAL(2,1),
    FOREIGN KEY (CustomerID)   REFERENCES
Customer(CustomerID)     ON UPDATE CASCADE,
    FOREIGN KEY (RestaurantID) REFERENCES
Restaurant(RestaurantID) ON UPDATE CASCADE
);
-- ON UPDATE CASCADE: if RestaurantID changes in
Restaurant, it changes
-- automatically in FoodOrder too.
```

```sql
INSERT INTO Restaurant VALUES (1, 'A2B', 'Chennai'),
(2, 'Murugan Idli', 'Madurai'),
                              (3, 'Dindigul Biryani',
'Chennai');
INSERT INTO Customer VALUES (1, 'Arun', 'Chennai'),
(2, 'Priya', 'Madurai');
INSERT INTO FoodOrder VALUES (1, 1, 1, 2, 300, 4.5),
(2, 2, 2, 3, 150, 5.0),
                             (3, 1, 1, 1, 200, 3.9);

-- Show cascade
UPDATE Restaurant SET RestaurantID = 10 WHERE
RestaurantID = 1;
SELECT * FROM FoodOrder;     -- orders 1 and 3 now
show RestaurantID 10

-- For each city: total orders, avg price, total
quantity where rating >= 4.2
SELECT r.City, COUNT(f.OrderID) AS TotalOrders,
       AVG(f.Price) AS AvgPrice, SUM(f.Quantity) AS
TotalQty
FROM FoodOrder f
JOIN Restaurant r ON f.RestaurantID = r.RestaurantID
WHERE f.Rating >= 4.2
GROUP BY r.City;

-- Restaurants with at least one order rated 5
SELECT * FROM Restaurant
WHERE RestaurantID IN (SELECT RestaurantID FROM
FoodOrder WHERE Rating = 5);

-- Restaurants that never received any order
SELECT * FROM Restaurant
WHERE RestaurantID NOT IN (SELECT RestaurantID FROM
FoodOrder);

-- New customer + order, COMMIT; update qty & price,
ROLLBACK
SET autocommit = 0;
START TRANSACTION;
INSERT INTO Customer VALUES (3, 'Kumar', 'Salem');
INSERT INTO FoodOrder VALUES (4, 3, 2, 2, 180, 4.0);
COMMIT;
START TRANSACTION;
UPDATE FoodOrder SET Quantity = 5, Price = 450 WHERE
OrderID = 4;
```

```
ROLLBACK;

-- Place order, savepoint, update rating, rollback to
savepoint
START TRANSACTION;
INSERT INTO FoodOrder VALUES (5, 2, 3, 1, 250, 4.0);
SAVEPOINT sp1;
UPDATE FoodOrder SET Rating = 2.0 WHERE OrderID = 5;
ROLLBACK TO sp1;
COMMIT;

-- Delete an order and ROLLBACK
START TRANSACTION;
DELETE FROM FoodOrder WHERE OrderID = 5;
ROLLBACK;

-- New customer, order, savepoint, another order,
roll back the second
START TRANSACTION;
INSERT INTO Customer VALUES (4, 'Divya', 'Trichy');
INSERT INTO FoodOrder VALUES (6, 4, 2, 2, 160, 4.8);
SAVEPOINT sp2;
INSERT INTO FoodOrder VALUES (7, 4, 3, 1, 220, 4.1);
ROLLBACK TO sp2;
COMMIT;
SET autocommit = 1;

-- GRANT / REVOKE
CREATE USER IF NOT EXISTS 'delivery_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'order_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS 'customer_user'@'localhost'
IDENTIFIED BY 'pass123';
CREATE USER IF NOT EXISTS
'restaurant_admin'@'localhost' IDENTIFIED BY
'pass123';
GRANT SELECT ON food.Restaurant TO
'delivery_user'@'localhost';
GRANT SELECT, INSERT ON food.FoodOrder TO
'order_user'@'localhost';
GRANT SELECT, INSERT, UPDATE ON food.Customer TO
'customer_user'@'localhost';
REVOKE INSERT ON food.FoodOrder FROM
'order_user'@'localhost';
GRANT ALL PRIVILEGES ON food.FoodOrder TO
'restaurant_admin'@'localhost';
```

```sql
-- VIEW
CREATE VIEW Food_Order_View AS
SELECT f.OrderID, c.CustomerName, r.RestaurantName,
f.Quantity, f.Price, f.Rating
FROM FoodOrder f
JOIN Customer c   ON f.CustomerID   = c.CustomerID
JOIN Restaurant r ON f.RestaurantID = r.RestaurantID;
SELECT * FROM Food_Order_View;
```

9. Stored Procedures And Functions (Mysql)

```
--
============================================================
====
-- 9. STORED PROCEDURES and FUNCTIONS (MySQL)
-- Simple table used by the "student"
procedures/functions
--
============================================================
====
DROP DATABASE IF EXISTS plsql;
CREATE DATABASE plsql;
USE plsql;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name      VARCHAR(50),
    Dept      VARCHAR(20),
    Marks     INT
);
INSERT INTO Student VALUES (1, 'Arun', 'CSE', 72),
(2, 'Priya', 'IT', 35), (3, 'Kumar', 'ECE', 58);

-- ---------- PROCEDURE: welcome message ----------
DELIMITER //
CREATE PROCEDURE Welcome()
BEGIN
    SELECT 'Welcome to the DBMS Lab!' AS Message;
END //
DELIMITER ;
CALL Welcome();

-- ---------- PROCEDURE: increase marks of a
specified student ----------
DELIMITER //
CREATE PROCEDURE IncreaseMarks(IN p_id INT, IN p_inc
INT)
BEGIN
    UPDATE Student SET Marks = Marks + p_inc WHERE
StudentID = p_id;
```

```
    SELECT * FROM Student WHERE StudentID = p_id;
END //
DELIMITER ;
CALL IncreaseMarks(1, 5);

-- ---------- PROCEDURE: display details of a student
by ID ----------
DELIMITER //
CREATE PROCEDURE GetStudent(IN p_id INT)
BEGIN
    SELECT * FROM Student WHERE StudentID = p_id;
END //
DELIMITER ;
CALL GetStudent(2);

-- ---------- PROCEDURE: delete a student record ----
------
DELIMITER //
CREATE PROCEDURE DeleteStudent(IN p_id INT)
BEGIN
    DELETE FROM Student WHERE StudentID = p_id;
    SELECT 'Student deleted' AS Message;
END //
DELIMITER ;
CALL DeleteStudent(3);
SELECT * FROM Student;

-- ---------- FUNCTION: square of a number ----------
DELIMITER //
CREATE FUNCTION Square(n INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN n * n;
END //
DELIMITER ;
SELECT Square(7) AS Result;

-- ---------- FUNCTION: add two numbers ----------
DELIMITER //
CREATE FUNCTION AddNumbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END //
DELIMITER ;
```

```sql
SELECT AddNumbers(10, 20) AS Sum;

-- ---------- FUNCTION: pass or fail ----------
DELIMITER //
CREATE FUNCTION PassOrFail(m INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    IF m >= 40 THEN
        RETURN 'PASS';
    ELSE
        RETURN 'FAIL';
    END IF;
END //
DELIMITER ;
SELECT Name, Marks, PassOrFail(Marks) AS Result FROM
Student;

-- Useful extras
SHOW PROCEDURE STATUS WHERE Db = 'plsql';
SHOW FUNCTION STATUS WHERE Db = 'plsql';
-- DROP PROCEDURE IF EXISTS Welcome;
-- DROP FUNCTION IF EXISTS Square;
```

10. Triggers (Insert / Update / Delete) In Mysql

```
--
============================================================
====
-- 10. TRIGGERS (INSERT / UPDATE / DELETE) in MySQL
-- Main table + a log (audit) table that the triggers
write into
--
============================================================
====
DROP DATABASE IF EXISTS trig;
CREATE DATABASE trig;
USE trig;

CREATE TABLE Account (
    AccountNo INT PRIMARY KEY,
    Name      VARCHAR(50),
    Balance   DECIMAL(10,2)
);
CREATE TABLE Account_Log (
    LogID      INT AUTO_INCREMENT PRIMARY KEY,
    AccountNo  INT,
    Action     VARCHAR(20),
    OldBalance DECIMAL(10,2),
    NewBalance DECIMAL(10,2),
    ActionTime DATETIME
);

-- ---------- AFTER INSERT trigger ----------
DELIMITER //
CREATE TRIGGER trg_after_insert
AFTER INSERT ON Account
FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (NEW.AccountNo, 'INSERT', NULL,
NEW.Balance, NOW());
END //
DELIMITER ;
```

```
-- ---------- AFTER UPDATE trigger ----------
DELIMITER //
CREATE TRIGGER trg_after_update
AFTER UPDATE ON Account
FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (NEW.AccountNo, 'UPDATE', OLD.Balance,
NEW.Balance, NOW());
END //
DELIMITER ;

-- ---------- AFTER DELETE trigger ----------
DELIMITER //
CREATE TRIGGER trg_after_delete
AFTER DELETE ON Account
FOR EACH ROW
BEGIN
    INSERT INTO Account_Log (AccountNo, Action,
OldBalance, NewBalance, ActionTime)
    VALUES (OLD.AccountNo, 'DELETE', OLD.Balance,
NULL, NOW());
END //
DELIMITER ;

-- Test them (this is the "implement" part)
INSERT INTO Account VALUES (1001, 'Arun', 5000);
INSERT INTO Account VALUES (1002, 'Priya', 8000);
UPDATE Account SET Balance = 6000 WHERE AccountNo =
1001;
DELETE FROM Account WHERE AccountNo = 1002;
SELECT * FROM Account_Log;

-- ---------- BEFORE INSERT trigger (validation
example) ----------
DELIMITER //
CREATE TRIGGER trg_before_insert
BEFORE INSERT ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SET NEW.Balance = 0;
    END IF;
END //
DELIMITER ;
```

```sql
INSERT INTO Account VALUES (1003, 'Kumar', -500);
SELECT * FROM Account;

SHOW TRIGGERS;
-- DROP TRIGGER IF EXISTS trg_after_insert;
```

11. Xml + Xsd Validation

File 1 · students.xsd (the rules)

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Root element <students> contains one or more
<student> -->
  <xs:element name="students">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="student"
maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name"
type="xs:string"/>
              <xs:element name="dept"
type="xs:string"/>
              <xs:element name="cgpa">
                <xs:simpleType>
                  <xs:restriction base="xs:decimal">
                    <xs:minInclusive value="0"/>
                    <xs:maxInclusive value="10"/>
                  </xs:restriction>
                </xs:simpleType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="id" type="xs:integer"
use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

File 2 · students.xml (the data)

```
<?xml version="1.0" encoding="UTF-8"?>
<students
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:noNamespaceSchemaLocation="students.xsd">
  <student id="101">
    <name>Arun</name>
    <dept>CSE</dept>
    <cgpa>8.5</cgpa>
  </student>
  <student id="102">
    <name>Priya</name>
    <dept>IT</dept>
    <cgpa>9.1</cgpa>
  </student>
</students>
```

File 3 · Validate.java

```
// Validate students.xml against students.xsd using
plain Java (no extra library)
// Compile: javac Validate.java     Run: java
Validate
import javax.xml.XMLConstants;
import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.*;
import java.io.File;

public class Validate {
    public static void main(String[] args) {
        try {
            SchemaFactory factory =
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA
_NS_URI);
            Schema schema = factory.newSchema(new
File("students.xsd"));
            Validator validator =
schema.newValidator();
            validator.validate(new StreamSource(new
File("students.xml")));
            System.out.println("XML is VALID
according to the XSD");
```

```
        } catch (Exception e) {
            System.out.println("XML is INVALID: " +
e.getMessage());
        }
    }
}
```

Or · validate.py (Python + lxml)

```
# Validate students.xml against students.xsd   (pip
install lxml)
from lxml import etree

schema = etree.XMLSchema(etree.parse("students.xsd"))
doc = etree.parse("students.xml")

if schema.validate(doc):
    print("XML is VALID according to the XSD")
else:
    print("XML is INVALID:")
    for error in schema.error_log:
        print(" -", error.message)
```

Run Java: `javac Validate.java` then `java Validate`. Break the XML on purpose (e.g. CGPA 12 or delete <name>) and run again to show "INVALID".